

Testing

Resumen

El testing es el proceso de verificar el software para asegurarse de que funcione correctamente. Cumpla con los requisitos, buscando y corrigiendo errores antes de que llegue al usuario.

El objetivo principal es asegurar la calidad y confiabilidad del producto, reducir costos y tiempo de desarrollo al detectar fallos temprano mejorando la experiencia del usuario.

En este proyecto lo hemos subdividido en dos secciones una para el Backend y otra para el Frontend

Herramientas utilizadas

Para el Backend:

- Postman
- Node con Newman
- Jenkins

Para el FrontEnd:

- Visual Studio Code
- Flutter Integration Test con Chrome Driver
- Jenkins

Desarrollo de Herramientas para el Backend

1. Postman

Es una plataforma integral que se utiliza en el proceso de desarrollo y testing. Para crear, probar, documentar y colaborar en el desarrollo de las APIs.

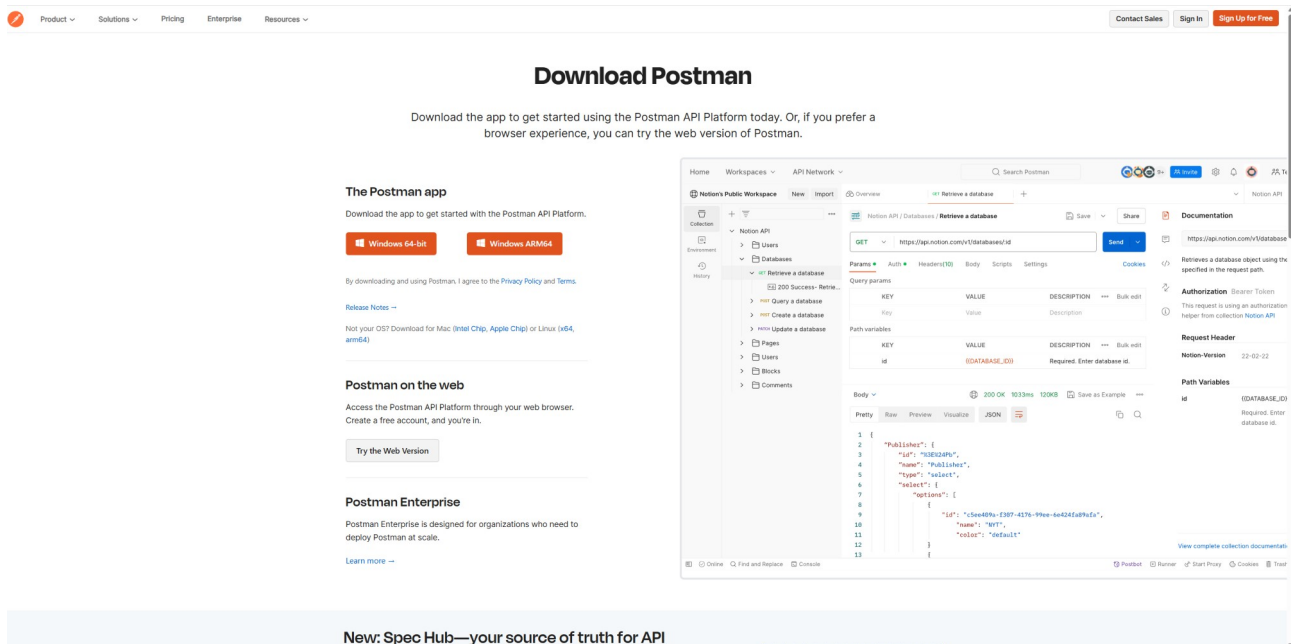
1.1. Funciones principales

- Realización de pruebas de las API realizando solicitudes http a los endpoint y visualizar las respuestas.
- Automatización de pruebas debido a que tiene la capacidad de crear y ejecutar pruebas(Runner Herramienta de ejecución) según un comportamiento esperado usando mocks(datos de pruebas) o directamente sobre la API
- Permite la colaboración desarrollador/tester para la posibilidad de documentación en conjunto. Compartiendo las consultas en un equipo o exportando en un fichero Json.
- Gestión de colecciones de solicitudes relacionadas.

1.2. Instalación de la herramienta

Accederemos a la pagina oficial de descarga desde un navegador:

<https://www.postman.com/downloads/>



Y descargaremos la version compatible con nuestro equipo. En este caso un equipo de windows 11 con versión 64-bit. El proceso de instalación seria el estandar sin configuraciones especiales. Para mas información del proceso de consultar la url oficial:

<https://learning.postman.com/docs/getting-started/installation/installation-and-updates/>

1.3. Configuración del entorno de pruebas

La configuración de este entorno orientado ha realizar consultas a los Endpoint de la API para localizar fallos en las respuestas (de mensajes tipo POST, PUT, PATCH, GET, DELETE) y poder prevenir fallos en caso de futuros desarrollo.

El backend se encuentra alojado en <https://tfg-backend-152779337859.europe-west1.run.app/> ruta que utilizaremos posteriormente junto al Swagger generado para la realizacion de las pruebas manuales durante el desarrollo con ruta

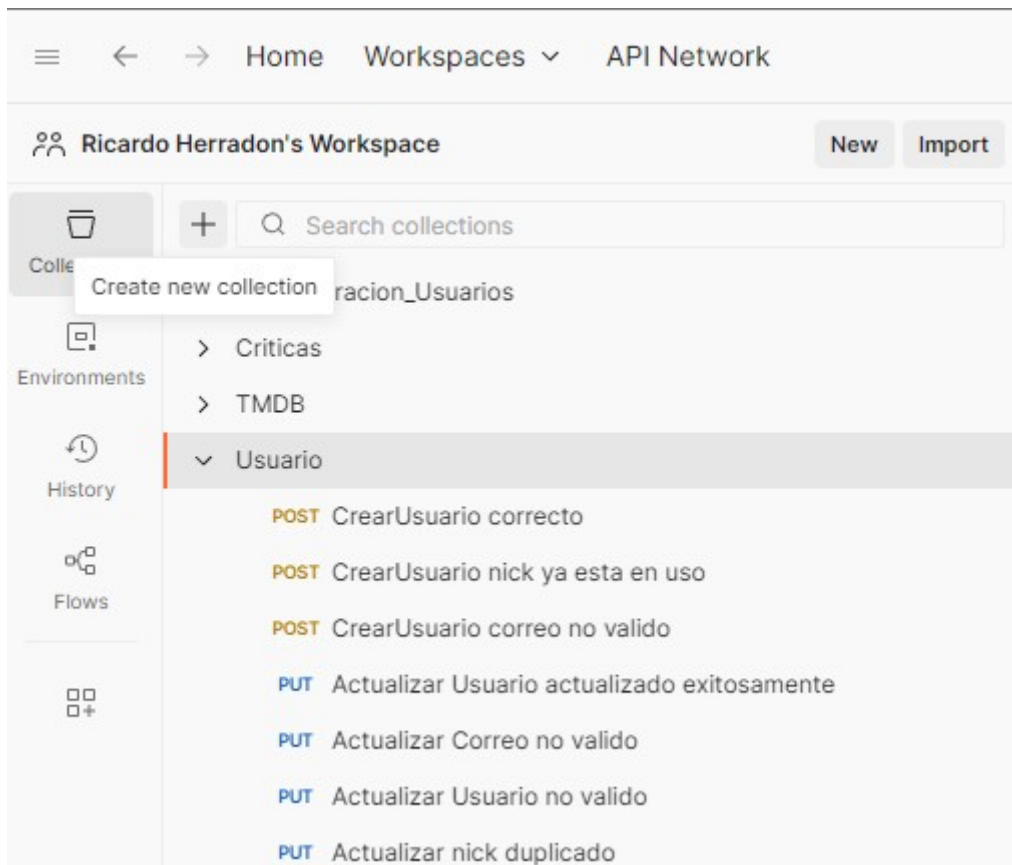
<https://tfg-backend-152779337859.europe-west1.run.app/swagger-ui/index.html#/>

En este proyecto tenemos los Endpoints agrupados en 4 secciones:

- **Administración de Usuarios Firebase:** Endpoints para la gestión de usuarios Auth de Firebase
- **Críticas:** Gestión de críticas y puntuaciones de películas
- **Usuarios:** Gestión de usuarios y sus perfiles
- **Búsqueda en TMDb:** Endpoints para buscar películas utilizando el servicio externo TMDb.

Configuración de colecciones según su sección:

Postman te permite agrupar tus consultas por **colecciones**, carpetas organizativas donde podemos crear las consultas necesarias.



Como se puede observar hemos creado una colección por cada sección de Endpoints. Dentro de las mismas es donde generaremos todas las consultas necesarias para verificar el correcto funcionamiento de los diferentes Endpoints.

Las consultas están compuestas de la siguiente estructura:

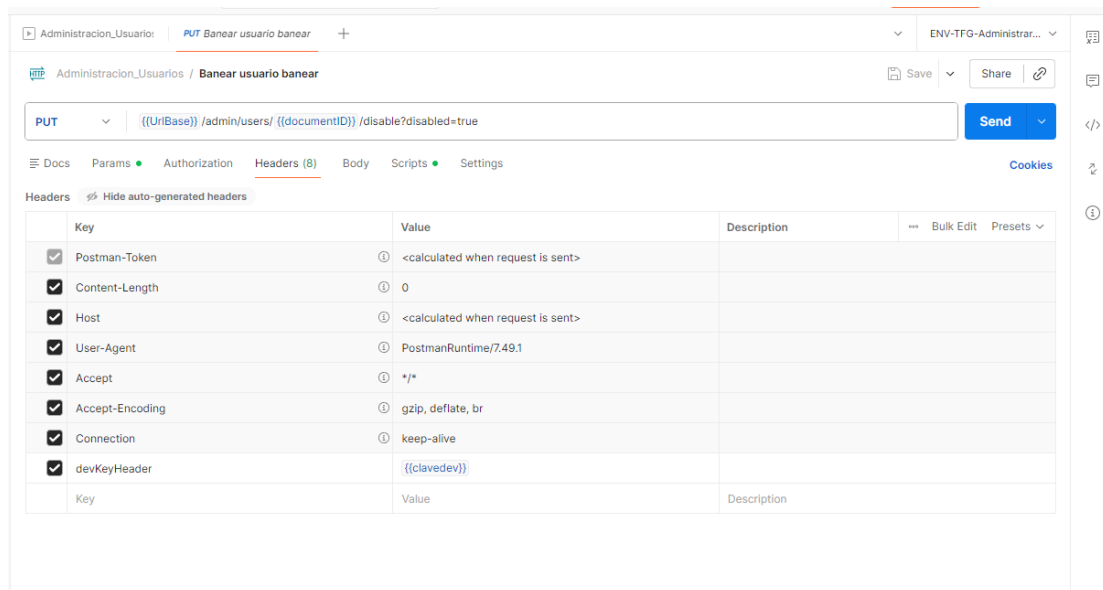
- **Header, cabecera del mensaje:** Son pares clave-valor para enviar información adicional con la solicitud, como el tipo de contenido (Content-Type) o datos de autenticación.
- **Ruta url del Endpoint:** Se trata de la dirección con la que interactuamos de la API, por ejemplo, URL base (<https://tfg-backend-152779337859.europe-west1.run.app/>) y ruta específica del Endpoint (/api/v1/usuarios/crearUsuario)
- **Parametros:** Se utilizan para enviar datos adicionales en una solicitud API, y se configuran principalmente en la pestaña **Params** o en el **Body(Cuerpo)**, dependiendo del tipo de parámetro. Ejemplo (urlImagen de tipo string, requerido por query)
- **Cuerpo:** Muchas veces se requiere de una gran cantidad de parametros y no es posible pasarlos con la solicitud como una cadena del mensaje. En este caso se desarrollo el cuerpo, una version ampliada que permite enviar datos de diferentes maneras seleccionando un tipo de cuerpo en la pestaña "Body", como form-data para campos y archivos, raw para texto plano, JSON, HTML o XML, y urlencoded para codificar los datos en la URL
- **Response, Respuesta de la consulta:** Es el resultado de una petición a una API.
- **Scripts:** Son fragmentos de código JavaScript que se pueden ejecutar en dos momentos: antes de la solicitud (**pre-request**) para configurar datos, o después de la respuesta (**post-**

response) para validar la respuesta. Permiten automatizar tareas repetitivas, definir variables dinámicas y realizar pruebas automatizadas en las API.

-

Ejemplo de consulta:

Header:



La peculiaridad de este encabezado es que es necesaria la utilización de una clave de desarrollador para realizar las funciones. **DevKeyHeader – {{clavedev}}**(Variable de entorno con la clave para que no este visible)

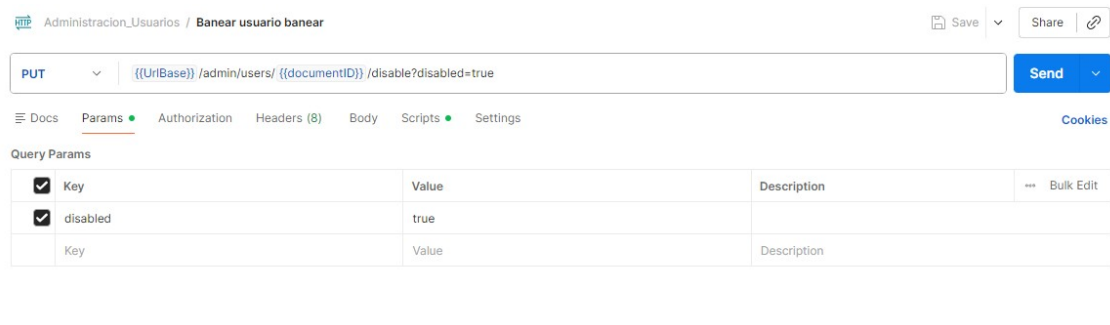
Ruta url del Endpoint:



Donde **URLBase** seria la url de nuestra API, **/admin/users/{{documentID}}/disable** la ruta especifica de nuestro endpoint y **?disabled=true** los parámetros a enviar por query.

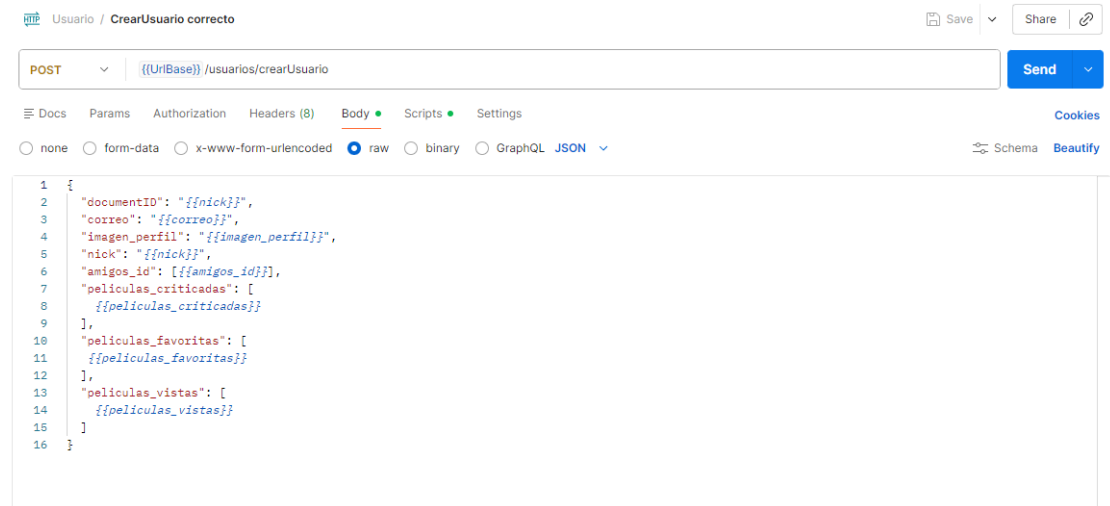
Parametros:

Ya hemos visto en el ejemplo anterior un parámetro en la url. Pero Postman te permite configurarlos de manera mas accesible desde la siguiente seccion.



Donde se puede configurar por pares de tipo clave-valor y lo interpreta añadiéndolo en la ruta URL.

Cuerpo:



Por ejemplo, en la creación de usuarios se requiere una serie valores pero estos generarían una ruta url muy larga ademas de exponer los datos en la misma consulta.

Para ello se utiliza el cuerpo donde nos permite agrupar por un formato especifico los valores a transmitir. En este caso el cuerpo esta compuesto por un fichero Raw. El cual esta configurado siguiendo las directrices de tipo JSON y compuesto por clave-valor(el valor utilizado son variables de entorno de postman que se sustituye por un valor preconocido o generado por los scripts).

Respuesta:



Toda las peticiones reciben una respuesta, en codigo de resultado 200 OK en este caso y dependiendo de la configuración del endpoint pueden devolver también un Cuerpo de mensaje.

Scripts:

Siguiendo con el ejemplo anterior, cuando se envía la petición y nos devuelve una lista de usuarios. Los scripts **post-response**, sirven para validar de manera automática si los datos de la respuesta.

The screenshot shows the Postman interface for a GET request to `{{UrlBase}}/usuarios`. The **Scripts** tab is active, displaying the following JavaScript code for the **Post-res** section:

```
1 pm.test("Tiempo de respuesta inferior a 200ms.", function () {
2   pm.expect(pm.response.responseTime).to.be.below(200);
3 });
4 pm.test("Codigo de respuesta 200, acceso correcto.", function () {
5   pm.response.to.have.status(200);
6 });
7
8 var json = pm.response.json();
9 pm.test("Se obtiene al menos un elemento de usuario, id: " + json[0].
10   documentID , function(){
11   if(typeof json[0].peliculas_vistas == "array"){
12     pm.expect(typeof json[0].documentID).to.be.equal("string");}
13 });
```

Below the script editor, the **Test Results** section shows the following outcomes:

- FAILED** Tiempo de respuesta inferior a 200ms. | AssertionError: expected 267 to be below 200
- PASSED** Codigo de respuesta 200, acceso correcto.
- PASSED** Se obtiene al menos un elemento de usuario, id: 3oGrjNxv6Q8TpD1MG6G4vwNQY22

En este caso validamos los siguientes datos: Que la respuesta sea mas rápida a 200ms , que el código de respuesta sea un 200 y que al menos contenga un array de objetos verificando que el primer elemento tiene un documentID ademas de mostrarlo como valor resultado del test.

Configuración de los entornos según su sección:

Postman te permite agrupar en **Entornos** las variables necesarias para la realización de las peticiones, generando una serie de elementos con clave-valor para a la hora de ejecutar las pruebas no tener que estar introduciendo los datos a mano.

En este proyecto lo hemos subdivido siguiendo el mismo orden que las colecciones:

- **ENV-TFG-Administrar-Usuario:** Variables para los Endpoints para la gestión de usuarios Auth de Firebase

- **ENV-TFG-Criticas-Controller:** Variables para la Gestión de críticas y puntuaciones de películas
- **ENV-TFG-Usuario-Controller:** Variables para la Gestión de usuarios y sus perfiles
- **ENV-TFG-TMDB:** Variables para los Endpoints para buscar películas utilizando el servicio externo TMDB.

Por ejemplo,

The screenshot shows the Postman Environments interface. On the left, a sidebar lists collections and environments. The 'ENV-TFG-Usuario-Controller' environment is selected and highlighted. The main panel displays a table of environment variables for this environment.

Variable	Value
UriBase	https://tfg-backend-152779337859.europe-west1.run...
documentID	Ricardo-Test
correo	Ricardo@ricardo.esedit
imagen_perfil	Ricardoedit
nick	RicardoPruebas
amigos_id	Ricardo,Ricardo,Ricardo
peliculas_criticadas	0,1,2,3
peliculas_favoritas	0,1,2
peliculas_vistas	0,1,2
nickedit	RicardoPruebas
documentIDotro	23l4EPjy0MpY4l4oeU6AGDOS8S2
Add variable	

1.4. Herramienta Runner

Runner es una herramienta de Postman que permite ejecutar manualmente una colección de peticiones en secuencia. Utilizada para probar y automatizar flujos de trabajo complejos, ya que puedes ejecutar todas las solicitudes de una colección una y otra vez, configurar el número de iteraciones o utilizar ficheros de datos(CSV) para verificar con diferentes datos, los resultados obtenidos.

Un ejemplo de ejecución sería el siguiente:

Selección de peticiones

The screenshot shows the Postman Runner interface. On the left, a sidebar lists collections. The 'TMDB' collection is selected, and a list of requests is shown. The main panel displays the 'Run Sequence' configuration for the selected collection.

Run Sequence

Run	Method	Request
1	GET	Consulta por nombre
2	GET	Consulta por nombre que no existe
3	GET	Consulta por nombre null
4	GET	Consulta por nombre vacío
5	GET	Consulta por id
6	GET	Consulta por id que no existe
7	GET	Consulta por id vacío
8	GET	Consulta por id null

Choose how to run your collection

- ☒ Run manually
- ☐ Schedule runs
- ☐ Automate runs via CLI

Run configuration

Iterations: 1

Delay: 0 ms

Test data file: Select File

Run TMDB

Ejecución de peticiones

The screenshot displays the Postman interface. On the left, a sidebar shows a collection named 'TMDB' under 'Environments'. The main panel shows the 'Run results' for the 'TMDB' collection. The results table includes columns for Source, Environment, Iterations, Duration, and All tests. The tests are categorized by HTTP method (GET) and endpoint (Consulta por nombre, Consulta por nombre que no existe, Consulta por nombre null, Consulta por nombre vacío, Consulta por id, Consulta por id que no existe, Consulta por id vacío, Consulta por id null). The results show that all tests passed, with a duration of 1s 442ms and 10 tests in total.

Source	Environment	Iterations	Duration	All tests
Runner	ENV-TFG-TMDB	1	1s 442ms	10

Avg. Resp. Time: 77 ms

All Tests: Passed (10) Failed (0) Skipped (0) [View Summary](#)

Iteration 1

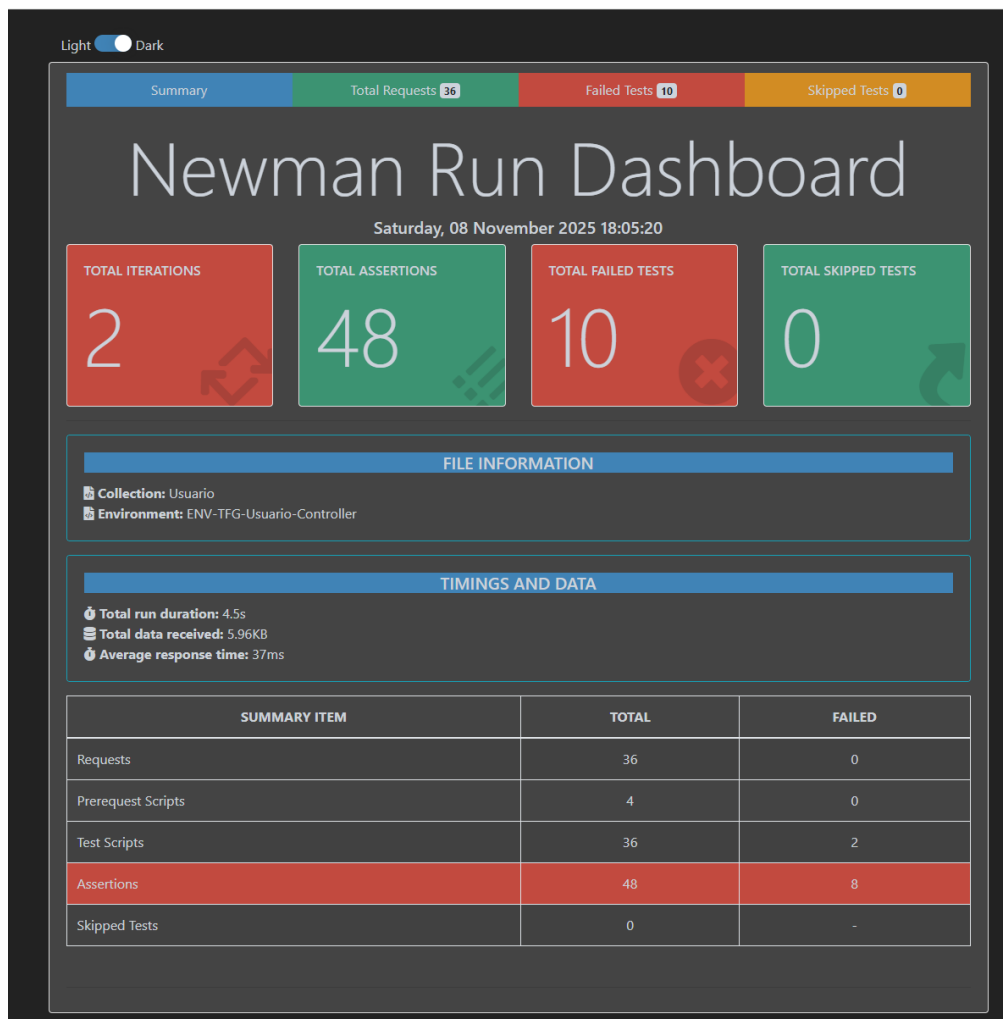
Test Name	URL	Status	Response Code	Response Time	Response Size	Test Result
GET Consulta por nombre	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorNombre...	PASS	200	141 ms	8.283 KB	2
GET Consulta por nombre que no existe	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorNombre?no...	PASS	200	65 ms	323 B	1
GET Consulta por nombre null	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorNombre...	PASS	200	175 ms	11.219 KB	2
GET Consulta por nombre vacío	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorNombre?no...	PASS	400	42 ms	457 B	1
GET Consulta por id	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorId?id=1...	PASS	200	83 ms	1.106 KB	1
GET Consulta por id que no existe	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorId?id=11111...	PASS	404	37 ms	374 B	1
GET Consulta por id vacío	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorId?id=	PASS	404	42 ms	374 B	1
GET Consulta por id null	https://ftg-backend-152779337859.europe-west1.run.app/tmdb/v1/peliculasPorId?id=null	PASS	404	38 ms	374 B	1

NEWMAM

Permite ejecutar y probar colecciones de Postman directamente desde la terminal, integrándose con pipelines de CI/CD para automatizar las pruebas de API sin necesidad de la interfaz gráfica.

Junto al modulo `newman-reporter-htmlextra` que permite la generacion de informes HTML para su facil lectura

Node con Newman: Node es un entorno de ejecucion de cofigo abierto y multiplataforma que permite ejecutar codigo en JavaScript. Newman es una herramienta de linea de comandos para la realizacion de pruebas exportadas desde Postman directamente en una consola de comandos. Ademas contiene funciones para la generacion de documentacion de las pruebas y Test generados en postman. Un ejemplo de comando seria el siguiente **“newman run Usuarios.postman_collection.json -e ENV-TFG-Usuario.postman_environment.json -d CSV-Iterador-Test-usuario.csv -r htmlExtra --insecure --folder newman/Usuario' bat 'xcopy newman/Usuario /TFG/proyectoTFG/Testing/Usuario /E /H”** Este comando generaría un informe en HTML para su posterior lectura.



Este informe indica detalladamente los datos que se han utilizado en la consulta de postman por ejemplo el cuerpo del mensaje enviado y el recibido.

Parte enviada.

REQUEST HEADERS

Header Name	Header Value
Content-Type	application/json
User-Agent	PostmanRuntime/7.39.1
Accept	*/*
Cache-Control	no-cache
Postman-Token	86566180-4a2b-4ca3-be56-f808cf9f5e75
Host	backend-proyectotfg-600260085391.europe-southwest1.run.app
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Content-Length	281

REQUEST BODY

```
{
  "correo": "Ricardo@ricardo.es",
  "imagen_perfil": "Ricardo",
  "nick": "Ricardo-test",
  "amigos_id": [
    "Ricardo",
    "Ricardo",
    "Ricardo",
    "Ricardo"
  ],
  "peliculas_criticadas": [
    0,
    1,
    2,
    3
  ],
  "peliculas_favoritas": [

```

Copy to Clipboard

Parte Recibida.

RESPONSE HEADERS

Header Name	Header Value
vary	Origin,Access-Control-Request-Method,Access-Control-Request-Headers
content-type	application/json
x-cloud-trace-context	8630143c7d134584724204d5388fc148;o=1
date	Sat, 08 Nov 2025 17:05:16 GMT
server	Google Frontend
Content-Length	249
Alt-Svc	h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

RESPONSE BODY

```
{
  "documentID": "atj3Hc2MahrRk0hK2A",
  "correo": "Ricardo@ricardo.es",
  "imagen_perfil": "Ricardo",
  "nick": "Ricardo-test",
  "amigos_id": [
    "Ricardo",
    "Ricardo",
    "Ricardo",
    "Ricardo"
  ],
  "peliculas_criticadas": [
    0,
    1,
    2,
    3
  ],
  "peliculas_favoritas": [

```

Copy to Clipboard

Batería de pruebas realizadas en esta consulta.

TEST INFORMATION						
			Search:			
Name	↑↓	Passed	↑↓	Failed	↑↓	Skipped
Codigo de estado 201, usuario creado		1		0		0
Usuario creado - documentID: xtj33HcXNWUhfR1oMAZA - nick: Ricardo-Test		1		0		0
Total		2		0		0

En caso de reportes erroneos hay un resumen con todos los campos y el motivo que se puede consultar desde el dashboard.

Light ☒ Dark

Summary

Total Requests **36**

Failed Tests **10**

Skipped Tests **0**

Expand All Failed Tests

SHOWING 10 FAILURES

Iteration 1 - AssertionError - Usuario - Consulta por ID error interno del servidor(Preguntar)

Failed Test: Codigo de respuesta 500, error interno.

ASSERTION ERROR MESSAGE

expected response to have status code 500 but got 200

Iteration 1 - AssertionError - Usuario - Consulta por Nick error interno(preguntar)

Failed Test: Codigo de respuesta 500, error interno.

ASSERTION ERROR MESSAGE

expected response to have status code 500 but got 200

Iteration 1 - AssertionError - Usuario - Eliminacion Error interno servidor(Preguntar)

Failed Test: Codigo de respuesta 500, Error interno

ASSERTION ERROR MESSAGE

expected response to have status code 500 but got 404

Instalación de herramienta

Node

Desde la pagina official de node en la url <https://nodejs.org/es/download>

Elegiremos el Sistema operativo del equipo en el que lo instalaremos, en nuestro caso un windows 11.

Newman

La instalación de Newman se realizara desde una consola de comando CMD en windows o Bash en Linux/MacOs con privilegios de administrador.

Utilizaremos el comando **npm install -g newman**

Modulo newman-reporter-htmlextra

Este modulo generara peticiones preconfiguradas exportadas desde postman.

Para su instalacion requiere de una consola de comando CMD en windows o Bash en Linux/MacOs con privilegios de administrador.

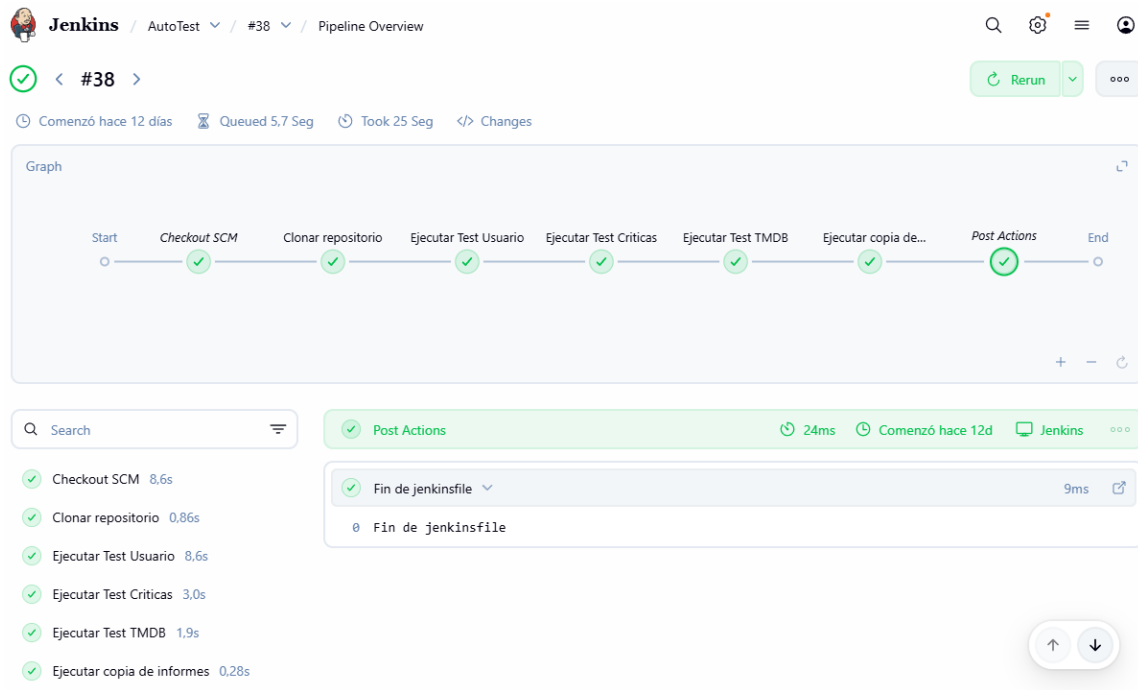
Utilizaremos el comando **npm i newman-reporter-htmlextra**

Jenkins: Es un servido de automatización de código abierto escrito en Java. Se utiliza para la integracion continua(CI) y entrega continua(CD) en el desarrollo de software y su mantenimiento. Sus principales características son:

- Automatización: Permite automatizar partes del desarrollo de software que son repetitivas y propensas a errores, como la compilación, las pruebas y el despliegue. Por ejemplo la clonación del repositorio en un servidor propio.

```
Started by GitHub push by RicardoHB996
Obtained Testing/jenkinsfile from git https://github.com/TFG-FlixScore/proyectoTFG.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\AutoTest
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> C:\Program Files\Git\cmd\git.exe rev-parse --resolve-git-dir
C:\ProgramData\Jenkins\.jenkins\workspace\AutoTest\.git # timeout=10
Fetching changes from the remote Git repository
> C:\Program Files\Git\cmd\git.exe config remote.origin.url https://github.com/TFG-FlixScore/proyectoTFG.git # timeout=10
Fetching upstream changes from https://github.com/TFG-FlixScore/proyectoTFG.git
> C:\Program Files\Git\cmd\git.exe --version # timeout=10
> git --version # 'git version 2.45.1.windows.1'
> C:\Program Files\Git\cmd\git.exe fetch --tags --force --progress -- https://github.com/TFG-FlixScore/proyectoTFG.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> C:\Program Files\Git\cmd\git.exe rev-parse "refs/remotes/origin/Testing^{commit}" # timeout=10
Checking out Revision b539048ca64278afdca0241cdbc2973d25051f0a (refs/remotes/origin/Testing)
> C:\Program Files\Git\cmd\git.exe config core.sparsecheckout # timeout=10
> C:\Program Files\Git\cmd\git.exe checkout -f b539048ca64278afdca0241cdbc2973d25051f0a # timeout=10
Commit message: "subida de ficheros actualizado"
> C:\Program Files\Git\cmd\git.exe rev-list --no-walk b93954111f42146c0f2e872f122fbe56fbe74127 # timeout=10
```

- **Integración y Entrega Continua (CI/CD):** La principal característica de este punto es implementar flujos de trabajo donde cada cambio en el código se compila, prueba y despliega de manera automática y continua. En nuestro caso aplica que cuando se genera un cambio en Github se lanzaría una serie de pruebas con Newman.
- **Expansibilidad:** Dispone de una enorme biblioteca de plugin desarrollados por la comunidad y para expandir sus funcionalidades. Por ejemplo conexión con github o con entornos IDE como Visual Studio Code.
- **Flexibilidad:** Dispone de acceso web adaptado a múltiples dispositivos.



- **Detección temprana de errores:** Al ejecutar pruebas de forma automatizada en cada cambio de código, los errores se detectan mucho antes en el ciclo de desarrollo.
- **Código abierto y comunidad:** Dispone de soporte continuo y cuenta con el apoyo de una extensa comunidad.

INTEGRATION TEST

Las pruebas de integration test verifican el flujo completo de la aplicación y la interacción entre componentes (widgets, servicios, bases de datos), llamadas end-to-end, simulando el uso real de un usuario en un emulador o dispositivo físico. Permitiendo a su vez generar unas pruebas que servirán para todos los entornos ya sea web, Windows, Android o IOS.

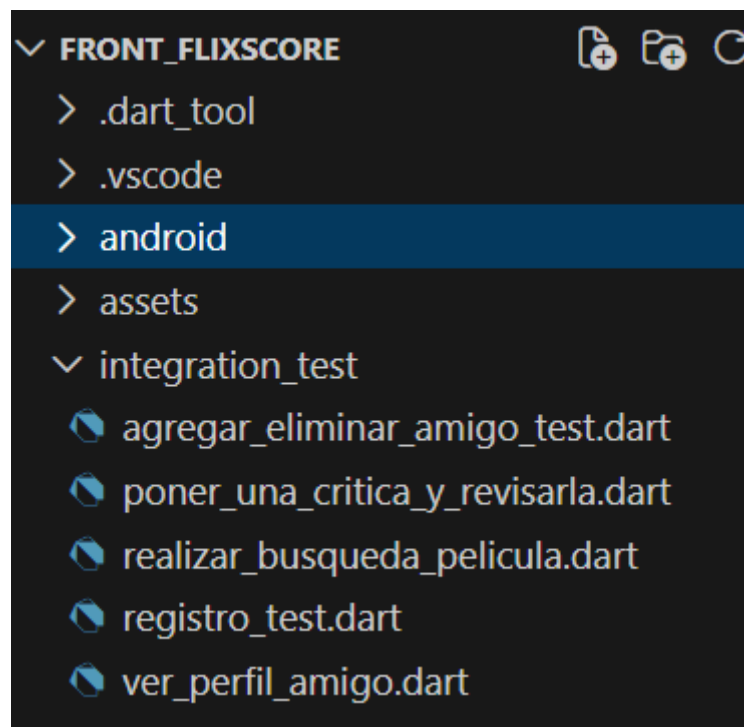
Estas pruebas requieren de acceso al código fuente desde el agente que vaya a realizar las pruebas.

En la raíz del proyecto se deben incluir las siguientes carpetas con los archivos .dart de las pruebas

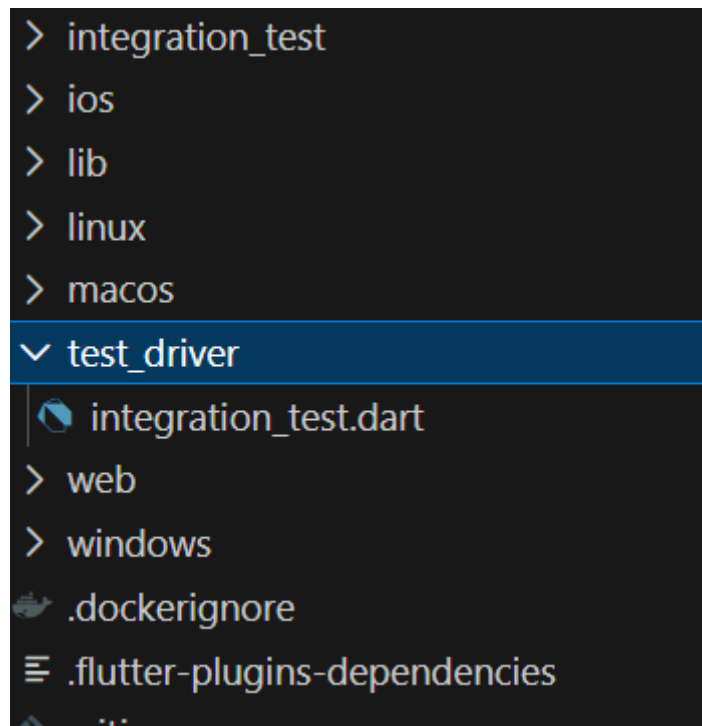
- Modificar el fichero pubspec.yaml: Agregaremos la dependencia integration_test

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  integration_test:  
    sdk: flutter
```

- Integration_test/: Generaremos la carpeta donde se ubicaran las diferentes pruebas
- Integration_test/ejemplo_test.dart: Los patrones de interacion con el GUI de la aplicación y sus correspondientes Aserciones para verificación del funcionamiento.



- Test_driver/integration_test.dart: Debido a que no es compatible sin la utilización de un driver en navegadores web, requiere de esta carpeta para la integración con chrome, firefox o edge.



Ejecución de pruebas:

Para la ejecución de pruebas en un navegador requiere de driver específico del navegador. En nuestro ejemplo lo haremos con chrome y su chrome driver.

Ejecutaremos en una consola el comando `chromedriver.exe --port=4444`

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Versión 10.0.26200.7171]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\ChromeDriver>chromedriver.exe --port=4444
Starting ChromeDriver 142.0.7444.175 (302067f14a4ea3f42001580e6101fa25ed343445-refs/branch-heads/7444@{#2749}) on port 4444
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully on port 4444.
```

Dejaremos el proceso del driver levantado.

Navegaremos con otra consola hasta la raíz de nuestro proyecto.

Y ejecutaremos el comando siguiente comando sustituyendo los dart por el de nuestro driver y la prueba a ejecutar.

```
flutter drive \ --driver=test_driver/integration_test.dart \ --target=integration_test/ejemplo_test.dart \
-d chrome
```

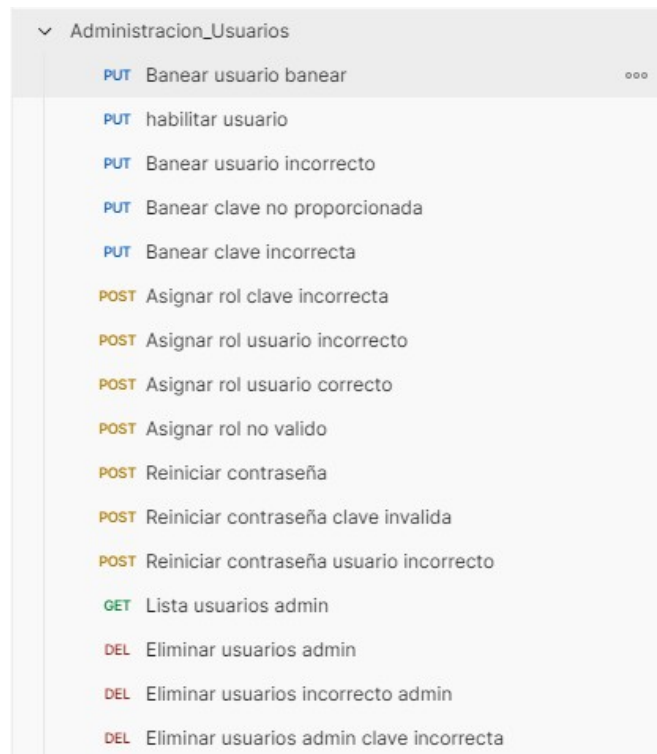
Esto lanzará un proceso de descarga de dependencias del proyecto y después establece una conexión con el chrome driver. Una vez establecido genera una ventana de navegador y realiza los pasos declarados en el `ejemplo_test.dart`

En la siguiente url podréis ver algunos ejemplos de ejecución de las pruebas: <https://github.com/TFG-FlixScore/proyectoTFG/tree/Testing/Testing/ejemplo%20de%20funcionalidad%20integration%20test>

1.5. Ejemplos peticiones postman

Colecciones y peticiones:

- **Administracion_Usuarios**



- **Criticas**

Criticas	
POST	CrearCritica exito
PATCH	Modificar Critica datos invalidos
PATCH	Modificar Critica document id incorrecto
PATCH	Modificar Critica puntuacion correcta
PATCH	Modificar Critica comentario correcta
POST	CrearCritica nulls
POST	CrearCritica error sin cuerpo
GET	Consulta todas las criticas
GET	Consulta filtrado por peliculaid
GET	Consulta filtrado por peliculaid valor null
GET	Consulta filtrado por peliculaid valor vacio
GET	Consulta filtrado por userid
GET	Consulta filtrado por userid valor null
GET	Consulta filtrado por userid valor vacio
GET	Consulta Recientes sin cantidad
GET	Consulta RecientesYDistintas sin cantidad
GET	Consulta Ranking sin cantidad
GET	Consulta Recientes cantidad 5
GET	Consulta Ranking cantidad 5
GET	Consulta RecientesYDistintas cantidad 5
GET	Consulta Recientes cantidad null
GET	Consulta Ranking cantidad null
GET	Consulta RecientesYDistintas cantidad null

- **TMDB**

TMDB	
GET	Consulta por nombre
GET	Consulta por nombre que no existe
GET	Consulta por nombre null
GET	Consulta por nombre vacio
GET	Consulta por id
GET	Consulta por id que no existe
GET	Consulta por id vacio
GET	Consulta por id null

- **Usuario**

▼ Usuario

- POST CrearUsuario correcto
- POST CrearUsuario nick ya esta en uso
- POST CrearUsuario correo no valido
- PUT Actualizar Usuario actualizado exitosamente
- PUT Actualizar Correo no valido
- PUT Actualizar Usuario no valido
- PUT Actualizar nick duplicado
- POST AgregarAmigo correcto
- POST AgregarAmigo usuario principal erroneo
- POST AgregarAmigo usuario amigo erroneo
- POST AgregarAmigo usuarios iguales Existentes
- POST AgregarAmigo usuarios iguales erroneos
- POST AgregarAmigo usuarios distintos erroneos
- GET Consulta por ID usuario encontrado exitosamente
- GET Consulta por ID usuario no encontrado
- PATCH Actualizar imagen url vacia
- PATCH Actualizar imagen correcto
- PATCH Actualizar imagen usuario incorrecto
- PATCH Actualizar nick vacio
- PATCH Actualizar nick duplicado
- PATCH Actualizar nick usuario no existe
- PATCH Actualizar nick correcto
- GET Consulta por Nick exitosa
- GET Consulta Amigos en comun correcta
- GET Consulta Amigos en comun ID principal erroneo
- GET Consulta Amigos en comun ID secundario erroneo
- GET Consulta Amigos en comun ambos ID erroneos
- GET Consulta todos los usuarios
- DEL Eliminacion amigo id principal erroneo
- DEL Eliminacion amigo id secundario erroneo
- DEL Eliminacion amigo id principal y secundario iguales
- DEL Eliminacion amigo id principal y secundario vacios
- DEL Eliminacion amigo correcta
- DEL Eliminacion exitosa
- DEL Eliminacion no existe usuario